Recommendation and Information Diffusion in a Social Network

SNTP Assignment 2 -- Epinions trust and rating data

Student	Muhammad Umar Afzaal
Roll number	23F-3106
Course	Social Network Analysis and Trust-based Prediction (SNTP)
Repository	https://github.com/muhammadumarafzaal/InformationDiffusion-SocialNetworks
Document	Searchable PDF generated from results/

This document reports an end-to-end study of recommendation and information diffusion on the Epinions trust network. All numbers were produced by the pipeline in this repository on Windows-10-10.0.19045-SP0 (4 logical CPUs, 15.7 GB RAM, Python 3.11.9) and are reproducible from the committed configuration and seeds.

Executive summary

The analysed Epinions core contains 10,706 users, 8,945 items, 300,304 ratings and 247,453 trust edges. Five recommenders, three social graph variants and two diffusion models were evaluated with a leakage-free protocol, and the full pipeline is reproducible from one configuration file.

- **Prediction.** bias_mf achieves the lowest test RMSE (1.0373), while item_knn produces the best ranking (NDCG@10 = 0.1384, hit rate = 0.3130). The two families therefore win on different objectives, which is exactly why both are kept.
- **Social signal.** The selected social weight is beta = 0.5000 (chosen on validation RMSE); on the test fold the strongest graph is 'similarity' (NDCG@10 = 0.0671). Friends-only prediction and the similarity graph are compared directly in Section 3, together with the ranking cost of the social pull.
- **Diffusion.** At p0 = 0.04 the Independent Cascade reaches 3,223.3 users (0.3011 of the core) and SIR reaches 5,406.1 users. SIR reaches further because an infectious user keeps retrying, but it needs roughly twice as many steps to get there; IC burns its single attempt immediately.
- **Seed selection.** At the tightest budget the informed strategies reach 23.6% more users than repeated random draws, while the marginal gain of an extra seed collapses after the first few picks; at this transmission rate the cascade saturates, and Section 5 reports that instead of hiding it.
- **Groups.** The three aggregation rules are compared on 60 groups of 5 members, and the rule choice is reported together with its agreement rate and its effect on realised spread (Section 6).
- **Intervention.** The harshest configuration tested removes 97.2% of the untreated reach but gives up 96.9% of the target audience; leaving exposure uncapped and gating conversion at 0.9 removes 15.9% for 5.5%. Both levers are compared on that trade-off rather than on reach alone (Section 8).
- **Engineering.** The vectorised neighbour gather is 12.6000x faster than the per-node loop, and is the rewrite the pipeline depends on. The gradient-accumulator rewrite manages only 1.1-1.3x, which is noise-level, and the report says so instead of presenting it as a speed-up (Section 8).
- **Audit.** The automated leakage audit reports all checks passing over fold integrity, candidate sets and campaign selection.

1. Problem statement and data (Requirement 1)

The task is to represent a social environment, learn users' preferences from it, extend the recommender with social relations, and then study how an item spreads through the same network under different diffusion models. Requirement 1 asks for a representation that separates what is *observed* from what is *created or inferred*. Table 1 draws that line explicitly.

quantity	raw epinions	analysed core
rating interactions	664,824.0	300,304.0
users	40,163.0	10,706.0
items	139,738.0	8,945.0
density	0.0001	0.0031
mean rating	3.9917	3.9109
median ratings per user	6.0000	15.0000
median ratings per item	1.0000	15.0000

Table 1. Raw Epinions versus the analysed core. The core is the result of an iterative k-core filter (users and items with at least 10 interactions).

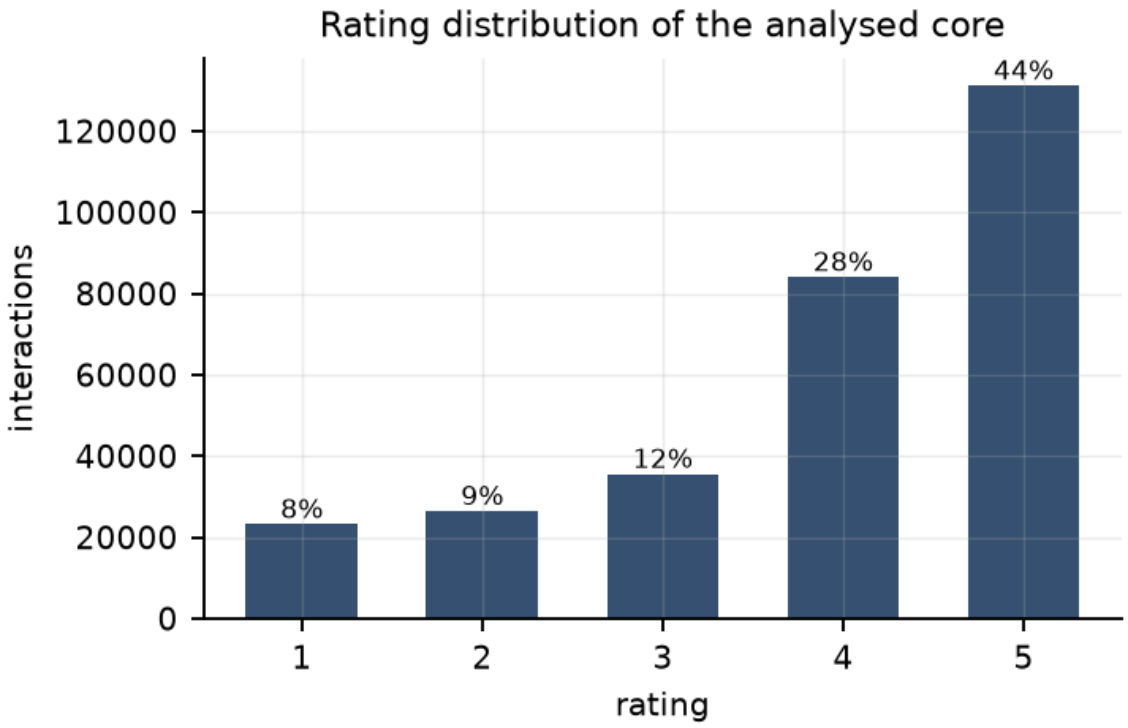


Figure 1. Rating distribution of the analysed core.

1.1 Observed versus created information

quantity	status	source
ratings (user, item, 1-5)	observed	Epinions ratings_data.txt, deduplicated
trust edges (source trusts target)	observed	trust_data.txt, all weights +1
influence edges (target -> source)	derived	direction reversed, no edge invented
train/validation/test folds	created	per-user random hold-out, fixed seed
ranking negatives	created	sampled never-observed items
taste similarity graph	inferred	bias-corrected profile cosine, trust edges removed
rewired control graph	created	degree-preserving target shuffle
campaign seeds, exposures, adoptions	simulated	never observed; reported separately

Table 1b. Every quantity used in the report, classified by whether it is observed in the dataset, derived deterministically from observations, created for evaluation, or simulated.

1.2 Network variants

Four graphs are used later. The trust graph is stored as published; the influence graph reverses every edge because information flows from the trusted user to the trusting user. The similarity graph answers the counterfactual 'what if the platform knew the same number of neighbours but chose them by taste?', and the rewired control keeps the degree sequence while destroying assortativity.

graph	nodes	edges	density	mean out degree	median out degree
trust (stored direction)	10,706	247,453	0.0022	23.1135	8.0000
influence (reversed)	10,706	247,453	0.0022	23.1135	5.0000
similarity (taste kNN)	10,706	107,004	0.0009	9.9948	10.0000
rewired control	10,706	239,857	0.0021	22.4040	5.0000

Table 2. Structural statistics of the four graph variants. (part 1 of 2)

graph	max out degree	isolated nodes	mean in degree	reciprocity	largest component fraction
trust (stored direction)	1,281	1,739	23.1135	0.4338	0.9441
influence (reversed)	1,677	841	23.1135	0.4338	0.9441
similarity (taste kNN)	10	0	9.9948	0.4490	1.0000
rewired control	1,272	841	22.4040	0.0269	0.9452

Table 2. Structural statistics of the four graph variants. (part 2 of 2)

fold	interactions	share
train	240,522	0.8009
validation	29,891	0.0995
test	29,891	0.0995

Table 3. Interaction folds. Validation is used for every tuning decision; the test fold is touched once per model.

1.3 Assumptions

- One interaction row = one review; duplicate (user, item) rows are removed.
- Ratings are explicit preference signals on a 1-5 scale; no implicit feedback is available.
- A trust record 'source target' means source trusts target; influence therefore flows target -> source.
- All trust weights in this release are +1, so distrust edges are absent and cannot be modelled.
- Epinions has no timestamps, so folds are random per user rather than temporal.
- Users and items with fewer than 10 interactions are pruned iteratively to obtain an evaluable core.
- Exposure and adoption events are simulated, never observed; they are reported separately.

The published ratings contain 487,002 trust records and 664,824 ratings before filtering. After the iterative 10-core the density rises from 0.0001 to 0.0031, which is what makes evaluation and simulation affordable on a single machine.

2. Learning preferences with two different recommenders (Requirement 2)

Requirement 2 asks for two substantially different approaches that both predict items the user has not seen, with at least one learning from the interaction data. Five models are trained so that the comparison is anchored: a global mean, a regularised bias model, a non-personalised popularity ranker, biased matrix factorisation and item-based k-nearest-neighbours. The last two are the two substantially different learners: one compresses users and items into a shared latent space, the other interpolates between items in the observed rating space.

2.1 Models

- **Global mean** predicts the training average everywhere: the floor.
- **Bias baseline** fits mu + b_u + b_i with ridge-regularised closed-form updates; it isolates how much of the signal is pure user/item generosity.
- **Popularity** ignores the user completely: every user receives the same row, and items are ranked by their shrunk mean rating (a Bayesian average), so a single lucky review cannot promote an item. It is the 'same list for everybody' control.
- **Bias-MF** learns p_u . q_i with 32 factors, mini-batch Adam, biases warm started from the ridge model, and early stopping on the validation fold. The optimiser, the gradient accumulation and the regularisation are implemented from scratch, without a recommender library.
- **Item-kNN** computes shrunk cosine similarities between bias-corrected item residuals, keeps the top 20 neighbours per item and clips the predicted correction to +/- 0.5 rating points.

The clip in item-kNN is worth explaining: neighbour interpolation has a long tail of weak evidence, and without a cap a single noisy neighbour can push a prediction to the edge of the scale. The cap costs a little accuracy on average but keeps the ranking well behaved, which is the objective that matters for a campaign.

2.2 Evaluation protocol

For each user the candidate list is every held-out test item plus 100 items sampled uniformly from the items the user has never rated in any fold. Relevance is binary: a test item counts when its observed rating is at least 4.0000. Candidates are shuffled before scoring, which removes the artefact that made constant-scoring models look perfect when positives were listed first. Ranking metrics and rating errors are computed from the same score matrix, so no metric uses a private scoring path.

2.3 Results

model	rmse	mae	precision@10	recall@10	hit@10	ndcg@10
Global mean	1.2618 +/- 0.0000	1.0014 +/- 0.0000	0.0193 +/- 0.0000	0.0967 +/- 0.0000	0.1714 +/- 0.0000	0.0551 +/- 0.0000
Bias baseline (ridge)	1.0418 +/- 0.0000	0.8157 +/- 0.0000	0.0303 +/- 0.0000	0.1555 +/- 0.0000	0.2528 +/- 0.0000	0.0945 +/- 0.0000
Popularity	1.0714 +/- 0.0000	0.8523 +/- 0.0000	0.0326 +/- 0.0000	0.1676 +/- 0.0000	0.2680 +/- 0.0000	0.1050 +/- 0.0000
Bias-MF (latent factors)	1.0373 +/- 0.0007	0.8056 +/- 0.0004	0.0255 +/- 0.0001	0.1295 +/- 0.0007	0.2163 +/- 0.0006	0.0748 +/- 0.0002
Item-kNN (neighbourhood)	1.0692 +/- 0.0000	0.8263 +/- 0.0000	0.0405 +/- 0.0000	0.2006 +/- 0.0000	0.3130 +/- 0.0000	0.1384 +/- 0.0000

Table 4. Test-fold performance, mean +/- standard deviation over three training seeds. (part 1 of 2)

model	map@10	coverage@10	diversity@10	novelty@10	gini@10	fit seconds
Global mean	0.0304 +/- 0.0000	0.6425 +/- 0.0000	0.9937 +/- 0.0000	13.3576 +/- 0.0000	0.5337 +/- 0.0000	0.0377 +/- 0.0315
Bias baseline (ridge)	0.0564 +/- 0.0000	0.1259 +/- 0.0000	0.9898 +/- 0.0000	12.7122 +/- 0.0000	0.9454 +/- 0.0000	0.1227 +/- 0.0328
Popularity	0.0646 +/- 0.0000	0.1289 +/- 0.0000	0.9900 +/- 0.0000	12.6179 +/- 0.0000	0.9453 +/- 0.0000	0.1200 +/- 0.0214
Bias-MF (latent factors)	0.0425 +/- 0.0003	0.1419 +/- 0.0004	0.9908 +/- 0.0000	13.1171 +/- 0.0013	0.9433 +/- 0.0000	65.9797 +/- 13.1855
Item-kNN (neighbourhood)	0.0918 +/- 0.0000	0.4134 +/- 0.0000	0.9901 +/- 0.0000	12.7464 +/- 0.0000	0.9041 +/- 0.0000	6.2847 +/- 1.7337

Table 4. Test-fold performance, mean +/- standard deviation over three training seeds. (part 2 of 2)

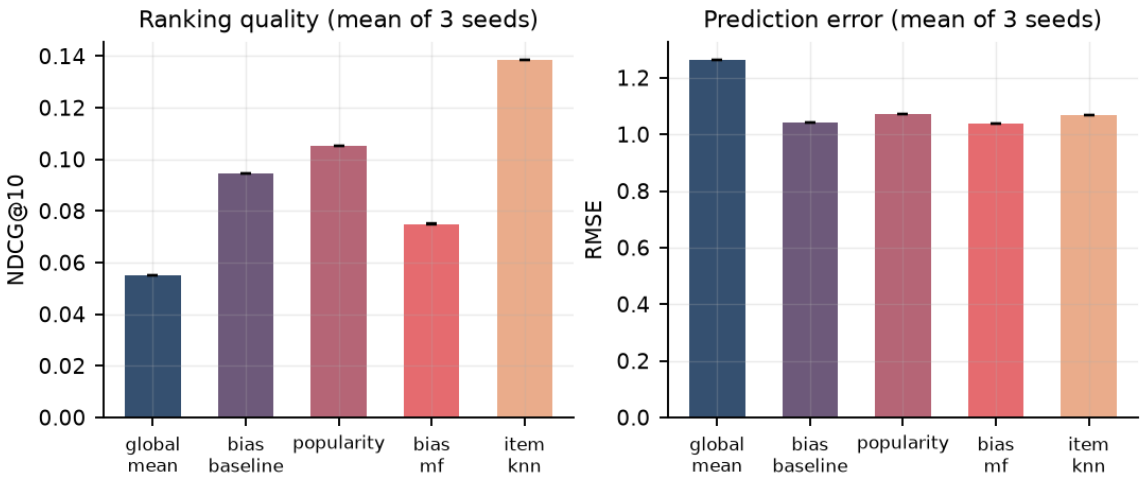


Figure 2. Ranking quality and prediction error of the five models.

Matrix factorisation gives the best RMSE (1.0373) and item-kNN the best ranking (NDCG@10 = 0.1384, hit rate = 0.3130). This is a genuine trade-off rather than a tuning failure: latent factors smooth over the whole interaction matrix and predict the rating level well, while a neighbourhood model reproduces the sharp item-to-item structure that decides which of a hundred candidates is worth opening. Coverage tells the same story, with item-kNN reaching 41.3% of the catalogue against 14.2% for Bias-MF.

2.4 Cold start

Requirement 2 asks specifically how behaviour changes for users and items with little evidence. Users are bucketed by training-history length and items by popularity quartile, and the same models are re-scored inside each bucket.

model	history bucket	users	ndcg	hit
Bias-MF (latent factors)	20-29	1,955	0.0773	0.2512
Bias-MF (latent factors)	30-49	1,168	0.0813	0.3459
Bias-MF (latent factors)	50-99	642	0.1029	0.5140
Bias-MF (latent factors)	<20	6,762	0.0672	0.1396
Bias-MF (latent factors)	>=100	179	0.1527	0.7933

Table 5. Bias-MF ranking quality by user history length.

model	history bucket	users	ndcg	hit
Item-kNN (neighbourhood)	20-29	1,955	0.1495	0.3719
Item-kNN (neighbourhood)	30-49	1,168	0.1674	0.5291
Item-kNN (neighbourhood)	50-99	642	0.1846	0.7009
Item-kNN (neighbourhood)	<20	6,762	0.1201	0.2064
Item-kNN (neighbourhood)	>=100	179	0.2404	0.8939

Table 6. Item-kNN ranking quality by user history length, on the same buckets.

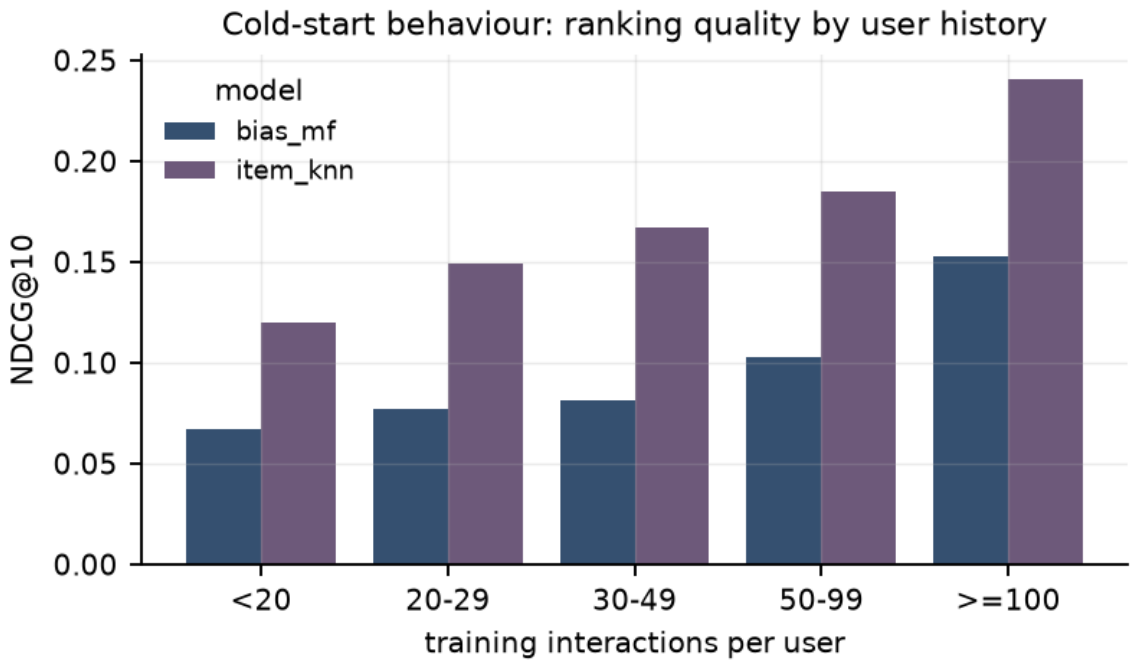


Figure 3. NDCG@10 by user history length for both learners.

model	popularity bucket	pairs	mae	rmse
Bias-MF (latent factors)	Q1 (coldest)	3,215	0.7714	0.9829
Bias-MF (latent factors)	Q2	2,864	0.7995	1.0287
Bias-MF (latent factors)	Q3	5,445	0.7804	1.0084
Bias-MF (latent factors)	Q4 (hottest)	18,367	0.8206	1.0574

Table 7. Bias-MF error by item popularity quartile (Q1 = coldest).

Ranking quality is a strong function of history: users with fewer than 20 training interactions are served far worse than users with a hundred or more. Error tells a more surprising story: cold items are *easier* to predict in terms of RMSE than popular ones, because their observed ratings cluster near the item mean that the bias model already learned. Cold start therefore hurts discovery quality much more than it hurts rating accuracy, which is why the campaign section uses ranking quality rather than RMSE to describe the system.

2.5 Accuracy for a well-defined classification task

model	accuracy	balanced accuracy	precision	recall	f1
Global mean	0.0195	0.5000	0.0195	1.0000	0.0383
Bias baseline (ridge)	0.1770	0.5231	0.0206	0.8831	0.0402
Popularity	0.1759	0.5250	0.0207	0.8882	0.0404
Bias-MF (latent factors)	0.1869	0.5261	0.0207	0.8791	0.0405
Item-kNN (neighbourhood)	0.1794	0.5162	0.0203	0.8667	0.0396

Table 8. Binary adoption task: given an unseen item, will the user rate it at least 4? Accuracy is reported next to the no-skill baseline because positives are rare. (part 1 of 2)

model	tpr	fpr	positive rate	no skill accuracy
Global mean	1.0000	1.0000	0.0195	0.9805
Bias baseline (ridge)	0.8831	0.8370	0.0195	0.9805
Popularity	0.8882	0.8383	0.0195	0.9805
Bias-MF (latent factors)	0.8791	0.8269	0.0195	0.9805
Item-kNN (neighbourhood)	0.8667	0.8343	0.0195	0.9805

Table 8. Binary adoption task: given an unseen item, will the user rate it at least 4? Accuracy is reported next to the no-skill baseline because positives are rare. (part 2 of 2)

Requirement 7 allows an accuracy figure only for a clearly defined classification task. The task here is: for a candidate item the user has never rated, predict whether the observed rating would be at least 4. Because positives are roughly two per hundred candidates, a classifier that always says 'no' already reaches about 98% accuracy; the table therefore reports balanced accuracy and the no-skill baseline next to the raw accuracy, and no ranking claim is based on accuracy.

3. Social relations: friends versus similar strangers (Requirement 3)

Requirement 3 asks to incorporate social relationships and to make the contribution of the social signal tunable. Two mechanisms are used. The first is a no-learning social predictor that replaces a user's residual for an item by the average residual of the people they trust. The second is a SocialMF-style regulariser added to biased matrix factorisation:

```
        L = L_rating + beta * sum_u || p_u - sum_{v in N(u)} s_uv p_v ||^2
        dL/dP(social) = 2 * beta * (E - S^T E), E = P - S P
```

S is the row-normalised influence graph, so every user is pulled towards the mean taste of the people they listen to, independent of how many they listen to. The gradient is injected through the same optimiser as plain biased MF, which makes beta = 0 an exact control rather than a re-implementation.

3.1 Do friends carry preference information?

model	graph	rmse	mae	ndcg@10	hit@10	precision@10	coverage@10
bias_baseline (no social)	none	1.0381	0.8060	0.0750	0.2159	0.0254	0.1416
neighbour_residual	trust	1.0947	0.8312	0.1226	0.3010	0.0371	0.5001
neighbour_residual	similarity	1.0684	0.8253	0.1172	0.2830	0.0343	0.3723
neighbour_residual	random	1.1058	0.8471	0.0901	0.2501	0.0295	0.4910

Table 9. Predicting residuals from friends only, with no latent factors (test fold).

The neighbour-residual predictor is deliberately weak in structure but interpretable: it has no latent factors and no learning beyond the bias model, so whatever it recovers comes from the graph. The similarity graph contains the same number of neighbours per user, chosen by taste, and every observed trust edge is removed from it before selection; the rewired graph keeps the degrees but randomises who is connected to whom.

Three things follow from the table. First, friends alone do not replace collaborative filtering: the trust graph scores 1.0947 against 1.0381 for the latent-factor model. Second, the identity of a neighbour does carry information, because the trust graph beats the degree-preserving rewired control (1.0947 against 1.1058). Third, and most directly relevant to the requirement, behaviour competes with the declared relationship: the taste-similarity graph, built only from training residuals, scores 1.0684 and therefore beats the friendship graph in this no-learning predictor. The SocialMF comparison in Section 3.3 does not reproduce that ordering for rating error, where trust wins, but it does for NDCG@10, so friends are informative without being uniquely so. Part of the gap here is coverage -- the trust graph leaves some users without a neighbour while the similarity graph covers everyone by construction.

3.2 How much social information helps

beta	rmse	mae	ndcg@10	hit@10	fit seconds
0.0000	1.0351	0.8038	0.0777	0.2243	62.2000
0.0500	1.0339	0.8018	0.0758	0.2217	105.2000
0.1000	1.0330	0.8004	0.0740	0.2173	82.8800
0.2500	1.0313	0.7966	0.0681	0.2048	75.2300
0.5000	1.0297	0.7935	0.0613	0.1909	86.8100
1.0000	1.0304	0.7937	0.0550	0.1763	90.9500
2.0000	1.0328	0.7963	0.0522	0.1693	93.2800

Table 10. Social weight swept on the validation fold; the test fold is untouched.

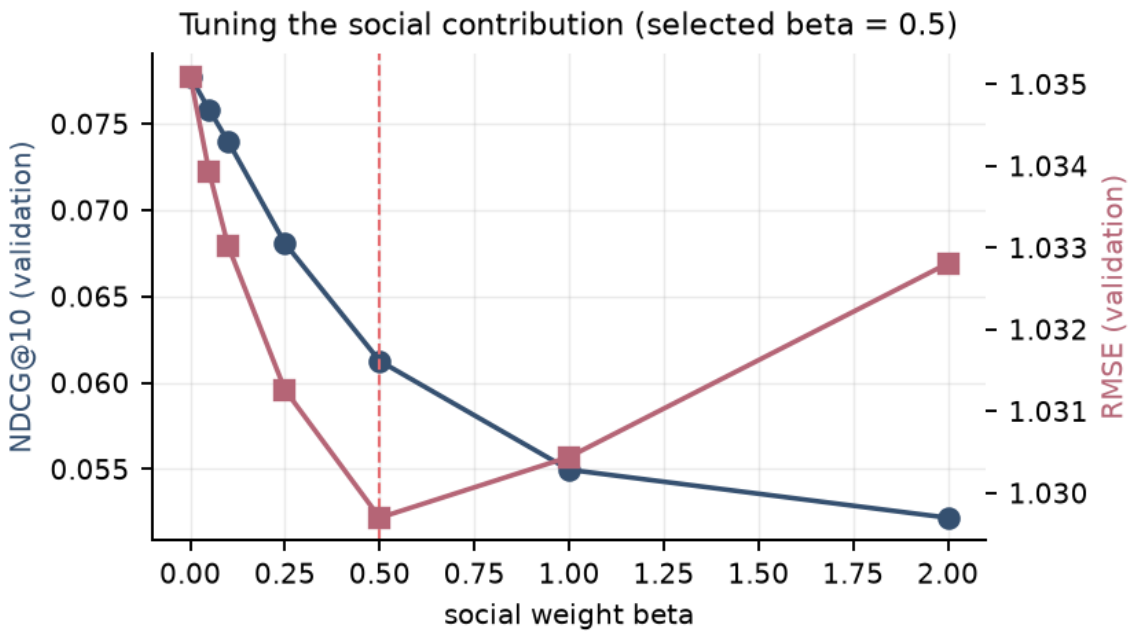


Figure 4. Validation NDCG@10 and RMSE as the social weight grows.

The selected weight is beta = 0.5000, chosen on the validation fold by the lowest rating error (RMSE), with ties broken towards the weaker social pull. RMSE is the objective the regulariser actually constrains, so selecting on it keeps the choice aligned with the mechanism; the test fold is never consulted. The sweep also shows the price of that choice: ranking quality falls monotonically as beta grows, from NDCG@10 = 0.0777 at beta = 0 to 0.0613 at the selected weight. Pulling users towards their neighbourhood averages their factors, which helps rating accuracy but removes exactly the idiosyncratic taste that separates a user's tenth-best item from their best. Large weights therefore flatten individual signal; the reported trade-off is the honest way to present it.

3.3 Friends versus similar-but-unconnected users

graph	beta	rmse	mae	ndcg@10	hit@10	precision@10	coverage@10
trust	0.5000	1.0328	0.7963	0.0572	0.1802	0.0208	0.1617
similarity	0.5000	1.0371	0.8024	0.0671	0.2018	0.0236	0.1527
random	0.5000	1.0370	0.8008	0.0554	0.1762	0.0202	0.1438

Table 11. The same social weight applied to the trust graph, the taste-similarity graph and the rewired control (test fold).

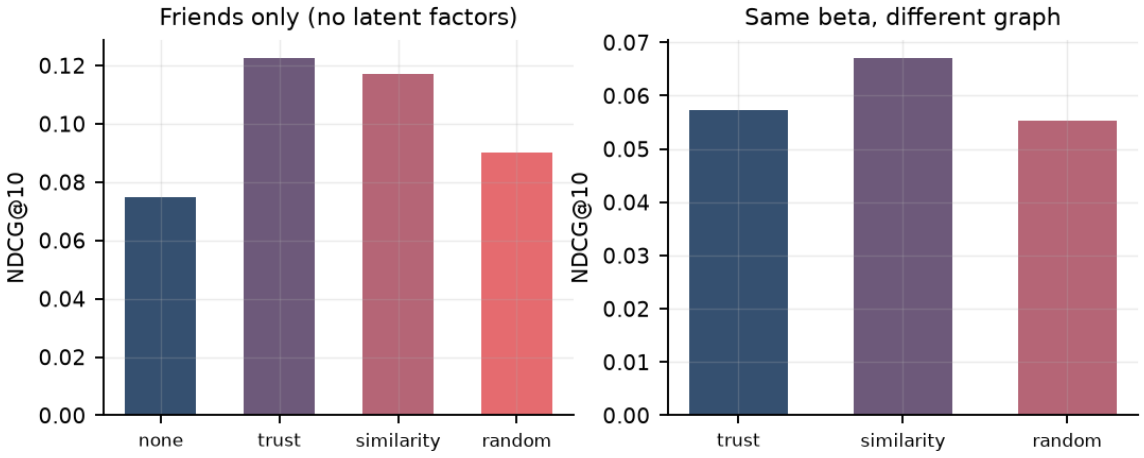


Figure 5. Friends-only prediction and the graph comparison.

metric	comparison	mean difference	ci low	ci high	significant	favours
mse	trust - similarity	-0.0036	-0.0085	0.0008	no	trust
mse	trust - random	-0.0054	-0.0100	-0.0010	yes	trust
ndcg@10	trust - similarity	-0.0099	-0.0114	-0.0083	yes	similarity
ndcg@10	trust - random	0.0018	0.0008	0.0030	yes	trust

Table 12. Paired bootstrap over per-user squared error and NDCG@10 differences; an interval that excludes zero is marked significant.

This is the central comparison of Requirement 3. If trust edges carried information that taste similarity does not, the trust graph should win; if the platform could do just as well by matching users on behaviour, the similarity graph should match it. The paired bootstrap makes the difference measurable at the level of individual users instead of comparing two averages. The rewired control shows how much of any gain is due to the degree distribution alone.

Against the rewired control the trust graph wins on both objectives: the per-user squared-error difference is -0.0054 [-0.0100, -0.0010] and the NDCG@10 difference is 0.0018 [0.0008, 0.0030], so the identity of a neighbour carries information that the degree sequence does not. Against the similarity graph the result splits by objective. Rating error still favours trust (-0.0036 [-0.0085, 0.0008]), but that interval includes zero, while NDCG@10 favours similarity (-0.0099 [-0.0114, -0.0083]) with an interval that excludes it. Declared friendship is therefore informative for the rating level, and behavioural matching is the better guide to which items to surface.

4. Adoption and information spread (Requirement 4)

A campaign promotes one item. The platform contacts a seed set directly; from there the item travels along influence edges. Three events are counted separately, as the requirement demands: **recommended** (a direct platform push), **exposed** (the item arrived from an adopting neighbour) and **adopted** (the user actually took it up). Two models are implemented.

- **Independent Cascade.** Every adopter gets exactly one transmission attempt per outgoing edge, in the step after adopting. Attempts are independent and never repeated.
- **SIR.** Infectious users attempt every susceptible neighbour once per step and recover with probability 0.35 per step. 'Recovered' means the user adopted but stopped spreading.

Transmission is not homogeneous. The probability of an edge succeeding depends on how much the receiver is predicted to like the item: p = p0 * (floor + (1 - floor) * affinity)^gamma * w, with gamma = 0.7000 and a floor of 0.1500. Setting the affinity weight to zero recovers the classical homogeneous models, which is the ablation reported below.

4.1 Campaign setup

The promoted item is the one with the highest mean predicted affinity among the 200 most-rated training items (internal id 2,152, original Epinions id 8,560). The launch set is the 20 users with the strongest predicted affinity, whose mean affinity is 1.0000 against a population mean of 0.9481. A seed still has to accept the recommendation; acceptance is probabilistic, so recommendation and adoption are not the same event.

4.2 Reach across transmission rates

model	p0	reach mean	reach std	reach fraction mean	steps mean	peak size mean
ic	0.0100	207.9500	94.0712	0.0194	13.6800	30.1200
ic	0.0200	1,553.5	75.2793	0.1451	15.0400	294.8100
ic	0.0400	3,223.3	59.1509	0.3011	11.7600	952.6500
ic	0.0800	4,807.9	43.7492	0.4491	9.3600	1,926.1
ic	0.1200	5,650.6	36.0630	0.5278	8.7500	2,535.4
sir	0.0100	2,354.2	99.2712	0.2199	30.0000	227.9000
sir	0.0200	3,971.7	85.9873	0.3710	29.8700	619.6500
sir	0.0400	5,406.1	59.7221	0.5050	29.2400	1,273.4
sir	0.0800	6,609.9	46.6789	0.6174	27.7300	2,100.8
sir	0.1200	7,202.1	42.8667	0.6727	26.9900	2,689.4

Table 13. Reach and timing for both models across the transmission grid (100 simulations per cell). (part 1 of 2)

model	time to 90pct mean	mean adoption step mean	attempts mean	seed adoptions mean	mean exposures mean	zero exposure users mean
ic	9.1900	4.6630	21,195.8	18.8600	1.9798	0.5890
ic	8.2000	5.3042	117,275.7	19.2400	10.9542	0.2914
ic	5.9700	4.0167	182,196.8	19.0700	17.0182	0.2262
ic	4.4900	3.2293	216,319.9	18.9700	20.2055	0.1985
ic	4.0000	2.9059	226,916.1	19.1700	21.1952	0.1887
sir	17.0200	10.3281	437,998.1	19.0200	40.9115	0.2518
sir	11.2000	6.8955	575,039.9	18.9800	53.7119	0.2110
sir	8.0200	5.0449	638,841.9	19.1600	59.6714	0.1913
sir	6.0100	3.9427	669,792.2	18.8500	62.5623	0.1804
sir	5.0600	3.4632	682,752.1	18.8600	63.7729	0.1766

Table 13. Reach and timing for both models across the transmission grid (100 simulations per cell). (part 2 of 2)

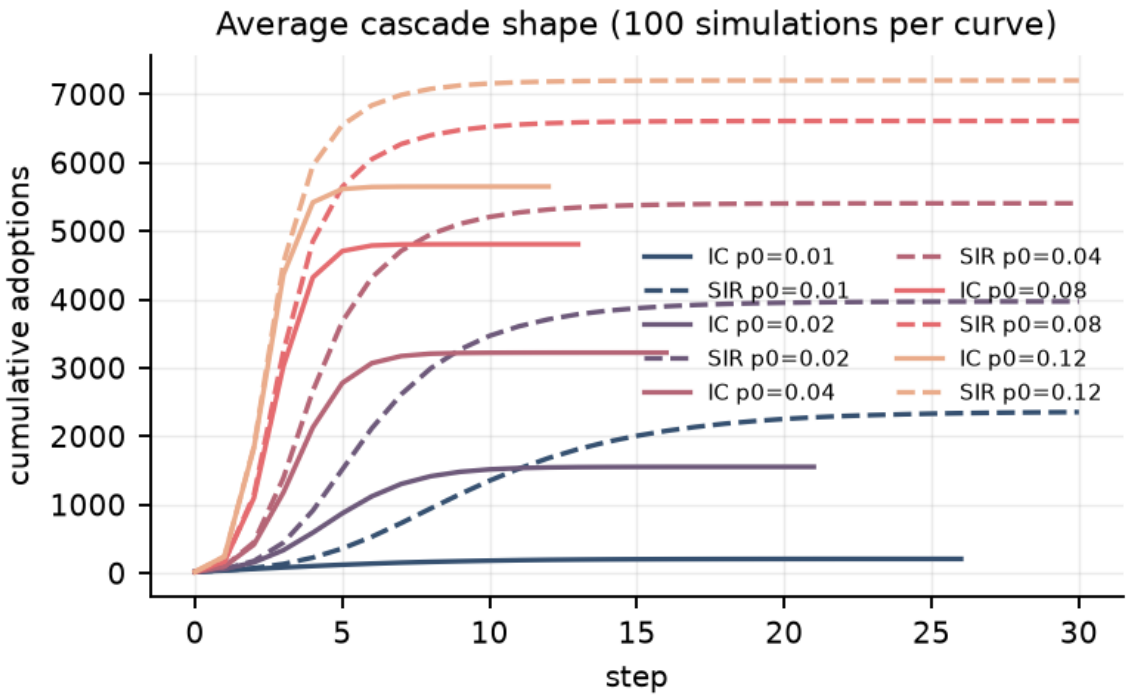


Figure 6. Average cascade shape per step and model.

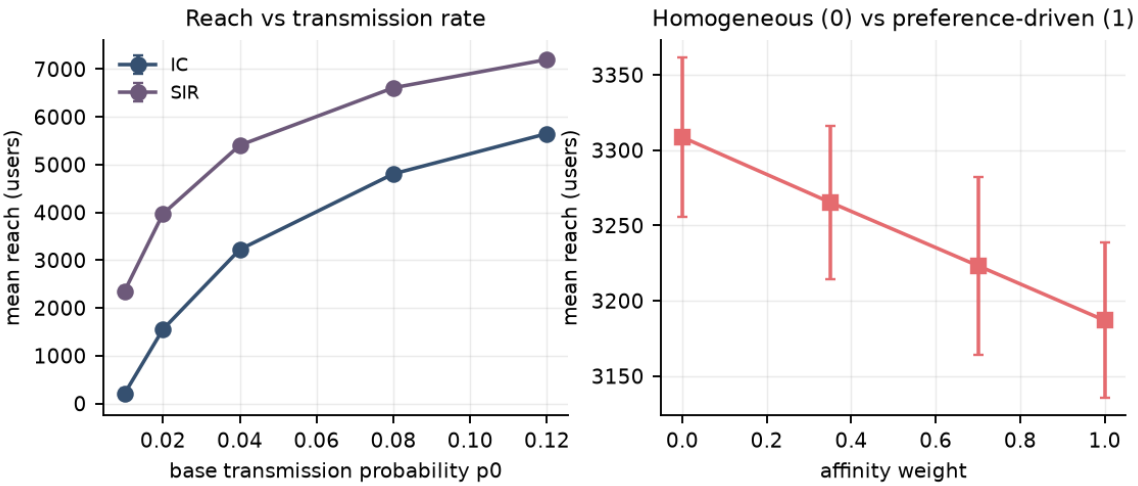


Figure 7. Reach versus transmission rate and affinity weight.

At very low p0 the cascade is near-critical: whether a run crosses the percolation threshold is itself a coin flip, so the run-to-run spread is proportionally far larger than at higher rates and the mean is a poor summary of any single campaign. As p0 grows the distribution concentrates. SIR overtakes IC quickly because an infectious user keeps trying, while IC gives up after one attempt; at high p0 both saturate most of the reachable component.

4.3 Affinity, recommendation and exposure

influence weight	reach mean	reach std	reach fraction mean	steps mean	peak size mean	time to 90pct mean
0.0000	3,308.6	52.9609	0.3090	11.6100	999.5400	5.8700
0.3500	3,265.4	50.9030	0.3050	11.6600	977.7100	5.9100
0.7000	3,223.3	59.1509	0.3011	11.7600	952.6500	5.9700
1.0000	3,186.9	51.5304	0.2977	11.6000	942.1700	5.9800

Table 14. Turning the recommender's affinity term off and on (IC, default p0). (part 1 of 2)

influence weight	mean adoption step mean	attempts mean	seed adoptions mean	mean exposures mean	zero exposure users mean
0.0000	3.9849	185,071.8	18.8400	17.2867	0.2238
0.3500	3.9766	183,572.2	18.8000	17.1467	0.2251
0.7000	4.0167	182,196.8	19.0700	17.0182	0.2262
1.0000	4.0504	181,400.2	18.7000	16.9438	0.2271

Table 14. Turning the recommender's affinity term off and on (IC, default p0). (part 2 of 2)

model	p0	recommended	seed adoptions	seed acceptance rate	adoptions from influence
ic	0.0400	20.0000	19.0700	0.9535	3,204.2
sir	0.0400	20.0000	19.1600	0.9580	5,387.0

Table 15. Recommended, exposed and adopted counts at the default transmission rate. (part 1 of 2)

model	reach mean	exposed users	share never exposed	conversion per exposure	attempts
ic	3,223.3	17.0182	0.2262	188.2820	182,196.8
sir	5,406.1	59.6714	0.1913	90.2771	638,841.9

Table 15. Recommended, exposed and adopted counts at the default transmission rate. (part 2 of 2)

The decomposition shows why the three events must not be conflated. The platform recommends the item to a small seed set; only a fraction accepts immediately; the rest of the reach arrives as exposures from neighbours, and only a fraction of those exposures convert. Reporting 'reach' as if it were 'recommendations delivered' would overstate the platform's direct influence by an order of magnitude.

5. Budgeted seed selection (Requirement 5)

The platform can contact only a fixed number of users. Choosing them well is a submodular maximisation problem: the value of a seed is the *additional* reach it creates on top of the seeds already chosen, not its individual popularity. The greedy search therefore evaluates marginal gain with Monte-Carlo cascades and uses CELF lazy evaluation to avoid recomputing candidates whose upper bound cannot beat the current best.

The candidate pool is the 300 users with the highest two-hop proxy score out of 9,084 eligible users. Four baselines compete on the same pool and the same simulator: the proxy itself, individual preference, degree, and repeated random draws.

5.1 Reach at equal budget

strategy	budget	reach mean	reach std	reach best	lift vs random
greedy_mc	5	3,212.5	52.8708	3,383.0	0.2365
proxy	5	3,233.8	53.9422	3,397.0	0.2447
preference	5	3,223.1	58.2029	3,365.0	0.2406
degree	5	3,227.9	49.2237	3,346.0	0.2424
random	5	2,598.1	746.9786	3,157.2	0.0000
greedy_mc	10	3,226.2	52.0295	3,367.0	0.1327
proxy	10	3,237.8	52.6727	3,375.0	0.1367
preference	10	3,226.4	52.6576	3,354.0	0.1328
degree	10	3,234.8	49.7786	3,365.0	0.1357
random	10	2,848.3	509.9207	3,225.3	0.0000
greedy_mc	20	3,232.3	50.5574	3,376.0	0.0087
proxy	20	3,240.6	46.6934	3,349.0	0.0113
preference	20	3,224.2	53.7641	3,363.0	0.0062
degree	20	3,240.6	46.6934	3,349.0	0.0113
random	20	3,204.4	41.1338	3,236.6	0.0000
greedy_mc	40	3,235.8	49.1030	3,361.0	-0.0020
proxy	40	3,244.8	49.8671	3,434.0	0.0008
preference	40	3,232.6	54.6245	3,400.0	-0.0030
degree	40	3,252.8	51.3575	3,425.0	0.0033
random	40	3,242.2	17.6629	3,255.3	0.0000

Table 16. Realised reach by strategy and budget, with the lift over the random baseline.

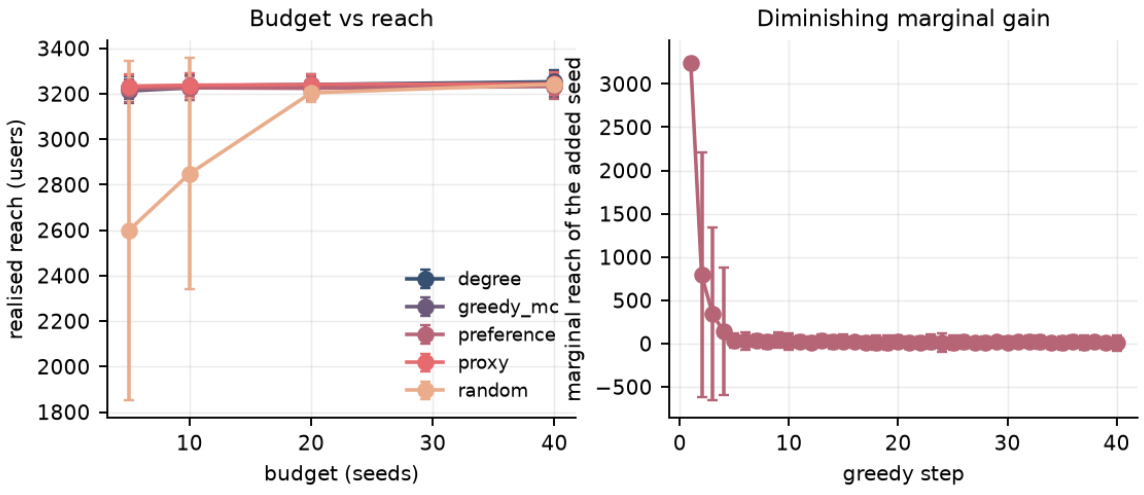


Figure 8. Reach versus budget and the marginal-gain curve.

The lift over random depends entirely on the budget. With five seeds the informed strategies reach about 24% more users than repeated random draws; with twenty or more, every strategy converges to the same value. The reason is visible in the cascade grid of Section 4: at the default transmission rate a single seed inside the giant in-component already percolates to roughly 3,200 users, so the budget stops mattering as soon as the seed set lands in that component. Random selection is worse precisely because it sometimes lands outside it, which is also why the spread

of its draws is an order of magnitude larger than that of any informed strategy. The four informed strategies are statistically indistinguishable from one another at every budget, so the marginal-gain machinery buys placement inside the giant component, not a better placement within it. The marginal-gain column makes the same point from the other side: the first pick is worth thousands of users, the last one 7.7.

5.2 Marginal gain and audience overlap

step	marginal gain	marginal std	cumulative gain
1	3,241.8	59.5190	3,241.8
2	790.2500	1,413.6	4,032.0
3	341.7000	997.5314	4,373.7
4	141.9500	739.7851	4,515.6
5	31.3000	82.6910	4,546.9
6	30.6500	98.3229	4,577.6
7	39.6500	72.2527	4,617.2
8	21.2000	78.4096	4,638.4
9	42.8500	90.2081	4,681.3
10	20.5500	94.0915	4,701.9
11	27.1000	70.4078	4,728.9
12	17.4000	66.6439	4,746.4
13	36.4000	68.1974	4,782.8
14	23.1000	76.7970	4,805.9
15	29.3500	78.8051	4,835.2
16	22.7000	53.5518	4,857.9
17	16.8500	71.7241	4,874.8
18	17.4000	82.7593	4,892.1
19	11.8500	85.6088	4,904.0
20	23.6000	73.2238	4,927.6
21	12.0500	68.0437	4,939.6
22	16.7000	71.3790	4,956.4
23	20.5000	83.4250	4,976.9
24	9.5500	107.8808	4,986.4
25	10.9000	87.7922	4,997.3
26	20.2500	79.6320	5,017.6
27	14.8500	45.2074	5,032.4
28	11.2500	76.1438	5,043.6
29	21.8500	51.8371	5,065.5
30	13.1000	65.8250	5,078.6

Table 17. Marginal reach contributed by each greedy pick, with the Monte-Carlo standard deviation.

strategy	sum audience	unique audience	audience overlap	mean pairwise jaccard	mean degree
greedy_mc	5,146.0	2,574.0	0.4998	0.0683	257.3000
proxy	13,777.0	4,542.0	0.6703	0.1233	688.8500
preference	2,260.0	1,299.0	0.4252	0.0471	113.0000
degree	13,777.0	4,542.0	0.6703	0.1319	688.8500
random	791.0000	686.0000	0.1327	0.0044	19.7750

Table 18. One-hop audience overlap of the seed sets at the median budget.

At this operating point neither cheap heuristic is correlated with measured single-seed reach: Spearman rho = -0.1537 for the two-hop proxy and -0.0847 for out-degree. That is the percolation effect again: inside the giant component almost every seed reaches the same audience, the measured reach of a candidate is nearly constant, and the ranks become noise. A heuristic can therefore prefilter the graph but cannot replace a simulation-based marginal gain. The overlap table shows the second thing greedy buys: its five seeds share only 50.0% of their one-hop audience, while the top-degree seeds share 67.0%.

5.3 Selection cost

method	candidates	budget	simulations	seconds	selected overlap with celf	speedup
naive greedy	30	3	1,200	8.8500	1	1.3687
CELF lazy greedy	30	3	848	6.4600	1	1.3687

Table 19. Exact greedy with and without lazy evaluation on an identical candidate pool.

The full greedy run used 32,440 cascades in 347.8994 seconds. Selection cost grows with the pool size times the budget, which is why the pool is prefiltered; the cost table quantifies what lazy evaluation buys on the same problem.

6. Group recommendation under conflicting preferences (Requirement 6)

A group of five users shares one decision. Three social-choice rules are compared on an identical candidate slate: **average** maximises the mean predicted satisfaction, **least misery** maximises the worst-off member's satisfaction, and **most pleasure** maximises the best-off member's. Groups are built in two flavours: friends (an anchor plus trust neighbours) and conflict groups (an anchor plus the most dissimilar users found in a random pool). Slates exclude everything any member has already rated in the training fold.

group kind	strategy	groups	mean satisfaction	min satisfaction	worst member
friends	average	60	4.8823	4.6631	4.0745
friends	least_misery	60	4.8816	4.6648	4.0745
friends	most_pleasure	60	4.8276	4.5605	4.0140
conflict	average	60	4.8952	4.7084	4.0745
conflict	least_misery	60	4.8947	4.7093	4.0745
conflict	most_pleasure	60	4.8293	4.5903	3.9568

Table 20. Satisfaction and realised spread by aggregation rule and group type. (part 1 of 2)

group kind	std satisfaction	reach	reach std	distinct items	consensus hits
friends	0.1275	3,166.2	232.5897	2	1.0000
friends	0.1271	3,166.9	224.6488	2	0.8667
friends	0.1648	3,135.1	257.1961	3	0.2167
conflict	0.1160	2,274.2	813.5309	2	1.0000
conflict	0.1161	2,281.3	807.8805	2	0.8500
conflict	0.1584	2,247.3	809.1511	3	0.0833

Table 20. Satisfaction and realised spread by aggregation rule and group type. (part 2 of 2)

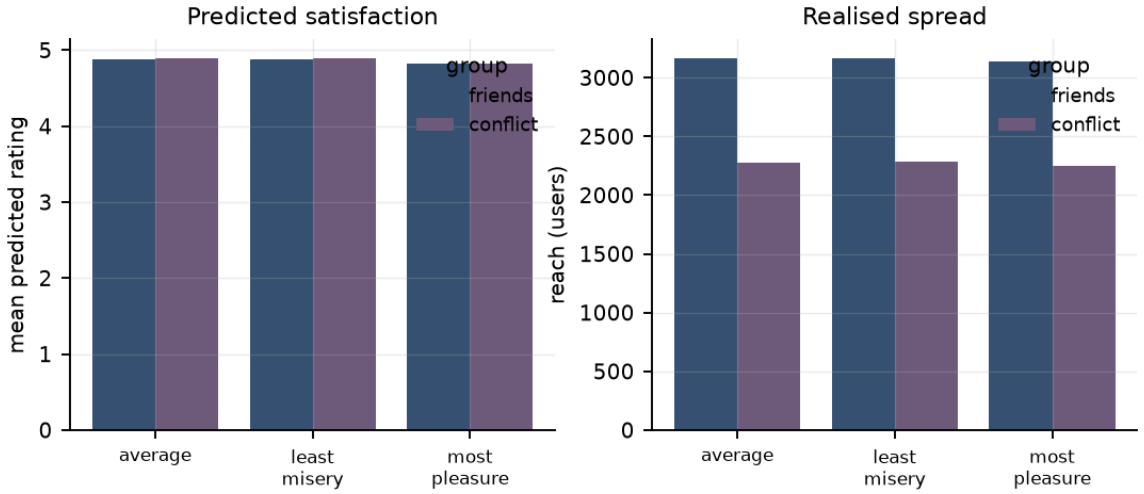


Figure 9. Predicted satisfaction and realised spread per rule.

6.1 Do the rules actually disagree?

group kind	pair	share same item	groups
friends	average vs least_misery	0.8667	60
friends	average vs most_pleasure	0.2167	60
friends	least_misery vs most_pleasure	0.2000	60
conflict	average vs least_misery	0.8500	60
conflict	average vs most_pleasure	0.0833	60
conflict	least_misery vs most_pleasure	0.0667	60

Table 21. Share of groups where two rules choose the same item.

Agreement is the honest way to report a group recommender: if the three rules usually pick the same item, the choice of rule hardly matters, and any claim about fairness is cosmetic. The table separates the two group constructions,

because a rule that only looks different on synthetic conflict groups has not demonstrated much.

Agreement is high where the rules should coincide and low where the group was built to disagree: average and least-misery return the same item in 86.7% of the friends groups, while average and most-pleasure agree in only 8.3% of the conflict groups. Between those two extremes the choice of rule is a real decision, and it is exactly the egalitarian-versus-utilitarian trade-off the requirement asks about.

6.2 Does the most satisfying item spread furthest?

group kind	spearman satisfaction vs reach	mean within group spearman	groups used within
friends	0.0379	0.6250	48
conflict	0.0012	0.2946	56

Table 22. Rank agreement between predicted satisfaction and realised reach.

The predicted satisfaction decides the item; the realised diffusion is simulated afterwards with the group as the seed set. The two quantities are therefore independent by construction. A weak or negative rank agreement means the item that makes the group happiest is not the item that travels furthest, which is precisely the tension a platform faces between satisfaction and virality.

Observed rank agreement. friends: rho=0.0379; conflict: rho=0.0012.

7. Evaluating the complete system (Requirement 7)

Requirement 7 asks for prediction error, precision, recall, ranking quality, coverage and diversity, diffusion outcomes, no leakage, repeated trials and an accuracy figure only where a classification task is well defined. Sections 2 to 6 cover the components; this section adds the guarantees that make those numbers trustworthy.

7.1 Leakage audit

check	result	detail
folds partition every interaction	PASS	300304 ratings split into 240522/29891/29891
test pairs never occur in training	PASS	0 overlapping pairs
validation pairs never occur in training	PASS	0 overlapping pairs
every user keeps training history	PASS	minimum 8 training interactions per user
campaign popularity uses the training fold only	PASS	0 items are cold in training and can never be promoted
ranking candidates exclude training items	PASS	0 violations over 63776 candidate slots (['test', 'valid'])
sampled negatives are never observed positives	PASS	every candidate is either a held-out test item or an item never observed in any fold

Table 23. Automated integrity and leakage checks. Each row is executed, not assumed.

The audit verifies that folds partition every interaction exactly once, that no test or validation pair appears in training, that every user keeps training history, and that ranking candidates contain no training item. It also records 0 items that are cold in the training fold (they can never be promoted) and 841 users without influence edges (they can be reached but cannot pass the item on).

7.2 Uncertainty and repeated trials

model	metric	mean	ci low	ci high	unit
bias_mf	ndcg@10	0.0750	0.0717	0.0782	user
bias_mf	hit@10	0.2159	0.2082	0.2235	user
bias_mf	precision@10	0.0254	0.0244	0.0264	user
bias_mf	rmse	1.0381	1.0289	1.0474	pair

Table 24. 95% percentile bootstrap intervals for the selected model on the test fold.

model	ndcg mean	ndcg std	rmse mean	rmse std	seeds	ndcg cv	rmse cv
bias_baseline	0.0945	0.0000	1.0418	0.0000	3	0.0000	0.0000
bias_mf	0.0748	0.0002	1.0373	0.0007	3	0.0030	0.0007
global_mean	0.0551	0.0000	1.2618	0.0000	3	0.0000	0.0000
item_knn	0.1384	0.0000	1.0692	0.0000	3	0.0000	0.0000
popularity	0.0945	0.0000	1.0753	0.0000	3	0.0000	0.0000

Table 25. Across-seed stability of every model, with coefficients of variation.

Every learned model is trained with three seeds and every diffusion configuration is run with a hundred cascades, so the reported means come with a spread. The bootstrap intervals resample users for ranking metrics and pairs for RMSE, which captures the uncertainty that a single train/test split leaves behind.

7.3 Calibration and observed versus simulated

bucket	count	mean prediction	mean observed	observed positive rate
(0.999, 2.991]	2,990	2.5245	2.3498	0.2244
(2.991, 3.386]	2,989	3.2073	3.1201	0.4423
(3.386, 3.653]	2,989	3.5280	3.4754	0.5724
(3.653, 3.857]	2,989	3.7593	3.8414	0.7029
(3.857, 4.023]	2,989	3.9425	4.0338	0.7655
(4.023, 4.171]	2,989	4.0962	4.1837	0.8193
(4.171, 4.316]	2,989	4.2444	4.2951	0.8531
(4.316, 4.465]	2,989	4.3895	4.4604	0.9010
(4.465, 4.657]	2,989	4.5568	4.6009	0.9341
(4.657, 5.0]	2,989	4.8178	4.7896	0.9709

Table 26. Predicted versus observed ratings in deciles of the prediction.

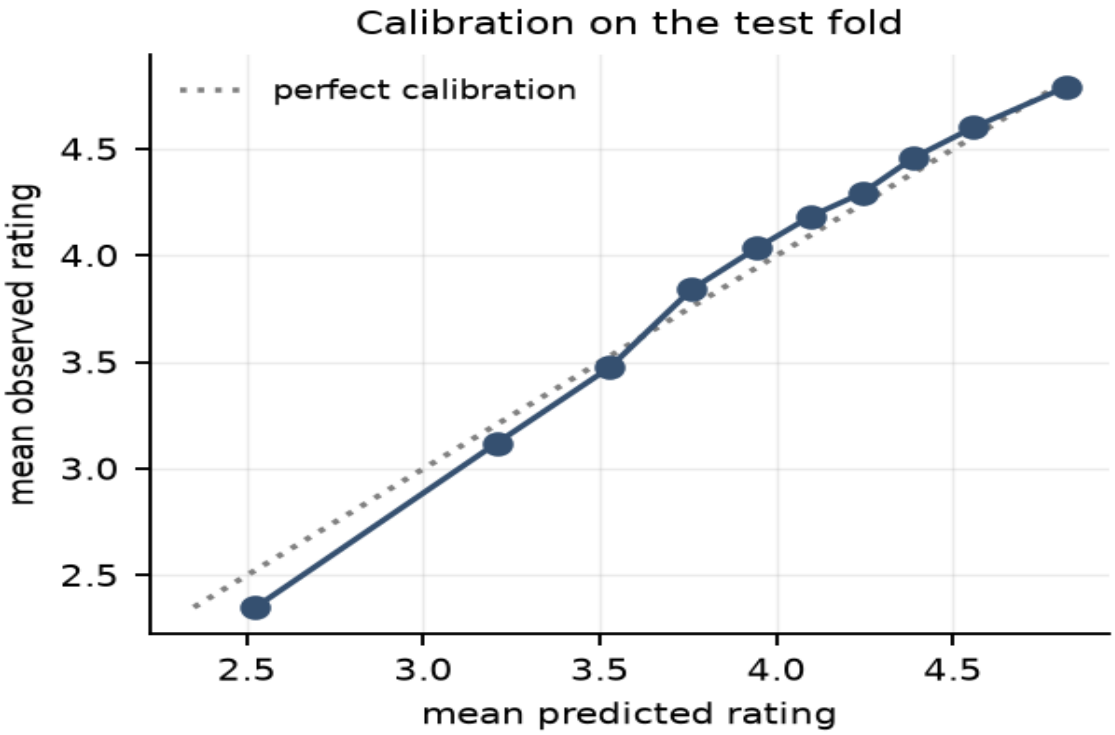


Figure 10. Reliability of the predicted rating scale.

The weighted mean absolute calibration error is 0.0769 rating points. The predicted range is compressed towards the middle, as it is for any regularised factor model: the bottom decile is over-predicted (the model cannot bring a rating below about 2.5) while the top decile is tracked closely. In ranking terms the compression costs nothing, because only the order inside a candidate set matters; in pricing or expectation terms it would matter.

item	original item id	train popularity	test popularity	mean affinity	simulated reach
69	363	1,202	144	0.4043	2,079.2
89	390	925	115	0.5851	2,562.5
161	615	752	87	0.8603	3,089.2
178	659	722	92	0.7315	2,888.4
179	660	638	99	0.7723	2,953.9
814	2,810	594	62	0.4335	2,150.4
2,017	7,611	557	77	0.2511	1,360.3
2,684	11,436	550	73	0.7979	2,977.9
301	1,083	517	73	0.8951	3,135.5
227	734	498	57	0.7560	2,902.8
324	1,109	477	57	0.9132	3,186.6
989	3,439	469	45	0.4507	2,252.0
2,703	11,550	453	54	0.5179	2,198.3
1,639	6,114	451	56	0.8793	3,124.2

item	original item id	train popularity	test popularity	mean affinity	simulated reach
1,646	6,128	431	52	0.8267	3,061.4
188	679	426	47	0.8747	3,131.9
1,732	6,509	425	58	0.8323	3,048.4
696	2,410	420	46	0.8960	3,154.0
891	3,017	409	31	0.4203	2,117.1
2,290	9,411	407	46	0.8221	3,037.6
1,202	4,036	387	56	0.7870	2,938.6
183	672	384	33	0.5795	2,541.7
90	391	379	37	0.8226	3,035.7
731	2,463	355	44	0.7973	2,732.8
640	2,294	345	65	0.6069	2,618.7
868	2,977	341	41	0.9321	3,190.9
202	700	322	51	0.4977	2,138.3
967	3,408	320	33	0.9250	3,186.9
2,363	9,758	319	40	0.0856	227.4000
1,048	3,573	313	47	0.8347	3,070.6

Table 27. Observed popularity versus simulated cascade size for thirty popular items.

Observed popularity is organic adoption accumulated over years; simulated reach is campaign-induced spread over a few steps. They are not the same quantity, and the report does not pretend otherwise: the rank correlation between them is -0.1382. The value of the comparison is diagnostic rather than predictive. A high correlation would mean the simulator is merely rediscovering popularity; the observed value shows it is driven by network position and affinity instead.

7.4 Diffusion outcomes

Diffusion is assessed with reach, reach fraction, number of steps, time to 90% of the final reach, peak frontier size, number of transmission attempts, mean exposure per user and the share of users never exposed. These are reported for both models in Section 4 and are the outcome metrics the requirement asks for.

8. Limiting diffusion and paying for it (Requirement 8)

Requirement 8 asks for an intervention that limits diffusion, a study of computational cost and scaling, and at least one measured optimisation. Two interventions act at different points of the pipeline: an exposure cap limits how many times one user can receive the same item, and an affinity gate refuses conversion to users whose predicted affinity is too low. Both run on the same seed set, item and random stream as the baseline, so the difference is causally attributable to the intervention.

exposure cap	affinity gate	reach mean	reach std	reach reduction	high affinity loss	reach high affinity	exposures mean
1	0.0000	103.3000	13.5635	0.9680	0.9676	30.5000	0.6917
1	0.7500	102.8600	10.9674	0.9681	0.9687	29.4800	0.6895
1	0.9000	90.7200	11.1320	0.9719	0.9688	29.3600	0.6501
2	0.0000	216.9800	22.4216	0.9327	0.9344	61.7400	1.1339
2	0.7500	218.4800	19.7482	0.9322	0.9327	63.3400	1.1583
2	0.9000	182.3700	17.1327	0.9434	0.9372	59.0400	1.0182
3	0.0000	333.9700	28.7801	0.8964	0.8972	96.7300	1.6469
3	0.7500	327.3000	25.1707	0.8985	0.8988	95.2100	1.6064
3	0.9000	277.8500	24.6160	0.9138	0.9019	92.3300	1.4329
1,000	0.0000	3,223.7	55.6432	0.0000	0.0000	940.7300	17.0761
1,000	0.7500	3,223.2	50.7724	0.0002	-0.0042	944.6500	17.0803
1,000	0.9000	2,709.9	60.6916	0.1594	0.0548	889.1500	15.0314

Table 28. Reach reduction against collateral damage among the high-affinity users the campaign actually targets.

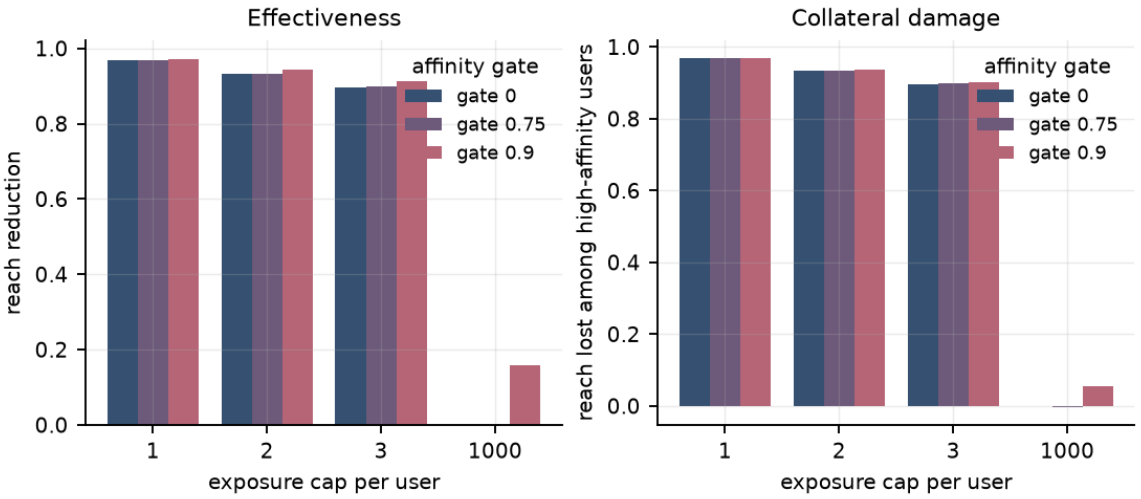


Figure 11. Effectiveness versus collateral damage.

8.1 Effectiveness versus collateral damage

The untreated campaign reaches 3,223.7 users. The strongest configuration tested (cap 1, affinity gate 0.9000) removes 97.2% of that reach while losing 96.9% of the high-affinity audience, which is the trade-off a platform has to sign off on.

The cheapest targeted lever looks different: leaving exposure uncapped and gating conversion at affinity 0.9 removes 15.9% of reach while losing 5.5% of the target audience, an order of magnitude less collateral damage than capping exposure. Which one a platform wants depends on whether the objective is to stop the campaign or to aim it.

The two levers act at different points. A cap of one exposure refuses reinforcement outright, so the cascade cannot build: reach collapses, but so does reach among the users the recommender was most confident about. A gate instead decides *who may convert at all*, so the campaign keeps travelling through the users it was aimed at and stops travelling through the rest. Comparing the two on the same table is the point: an intervention that only reports how much reach it removed is not an evaluation, because removing all reach is trivially possible.

8.2 Computational cost and scaling

users	ratings	trust edges	fit seconds	best epoch	rss delta mb	serve ms per 512 users	ratings per second
2,000	63,111	10,394	8.8700	12	9.4000	97.9900	85,345
4,000	126,155	43,038	15.5900	8	0.4000	121.9600	64,737
8,000	249,478	171,732	24.3300	6	2.3000	151.6400	61,513

Table 29. Recommender training cost against the number of users (real sub-samples).

items	users per batch	score ms	scores per second	dense matrix mb	intrinsic dim
1,000	512	13.5200	37,875,201	1.9500	32
3,000	512	55.4700	27,689,625	5.8600	32
5,500	512	93.9100	29,987,236	10.7400	32

Table 30. Serving cost against catalogue size for one batch of users.

users	edges	mean degree	seeds	ms per simulation	ms per million edges
2,000	39,893	19.9465	40	2.9700	74.3700
8,000	159,506	19.9383	160	36.8500	231.0300
20,000	399,961	19.9980	400	10.7700	26.9300

Table 31. Cascade cost against graph size on synthetic graphs with comparable mean degree.

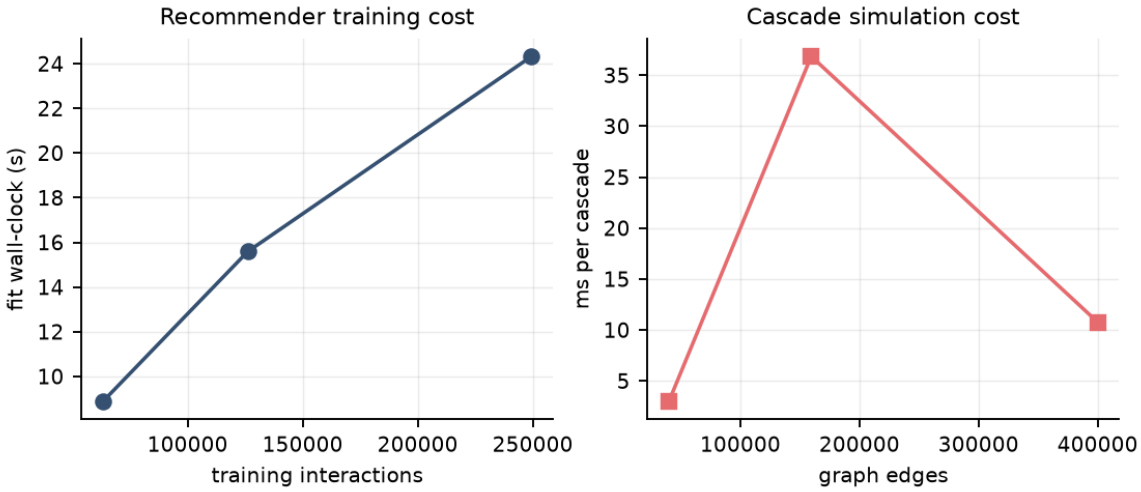


Figure 12. Training cost versus interactions and cascade cost versus edges.

Training cost grows with the number of interactions, not with the catalogue: each epoch touches only the observed pairs, and the run stops at its best validation epoch, so doubling the users does not double the wall-clock time. Serving cost is linear in catalogue size and in the number of users scored, which is why the evaluation streams users in chunks instead of materialising the full score matrix. Cascade cost grows with the number of edges traversed, so the simulation scales with the active frontier rather than with the whole population.

8.3 Measured optimisations

operation	baseline	baseline ms	optimised	optimised ms	speedup
gather 2000 neighbour lists (IC step)	per-node Python loop	10.2800	vectorised ragged gather	0.8200	12.6000
accumulate 1024x32 gradients into 900 rows	np.add.at	1.5800	sort + np.add.reduceat	1.2500	1.3000
accumulate 4096x32 gradients into 900 rows	np.add.at	4.1900	sort + np.add.reduceat	3.9000	1.1000

Table 32. Two hot-path rewrites measured on the real graph and on real batch sizes; the group-sum rows report what was measured rather than what was hoped for.

Optimisations are measured here, not claimed. The neighbour gather replaces a per-node Python loop with a ragged vectorised index, and the gradient accumulator replaces `np.add.at` with a sort followed by `np.add.reduceat`.

The first is what makes a hundred-cascade Monte-Carlo budget affordable. The second is the more instructive case, because sorting only pays off once a batch is large; it is benchmarked at the batch the trainer actually uses and at four times that batch.

Measured effect. The vectorised gather is 12.6000x faster than the per-node loop (10.2800 ms to 0.8200 ms per call), and it is the optimisation the pipeline depends on.

The group-sum rewrite is the instructive case rather than a headline result: 1.3000x for accumulate 1024x32 gradients into 900 rows; 1.1000x for accumulate 4096x32 gradients into 900 rows. A sorted reduction is never the bottleneck at these sizes, so the measured gains are small enough to sit inside run-to-run noise and do not grow with the batch as the rewrite originally assumed. It is kept because it makes the gradient code readable, and it is reported as a marginal change rather than a speed-up; the gather is the optimisation that earned its place.

The whole intervention study finished inside 139.7000 MB of resident memory on a graph with 247,453 edges.

9. Limitations

- **No timestamps.** Epinions as distributed here has no review dates, so the hold-out is random per user instead of temporal. A temporal split would be a harder and more realistic test of whether the model generalises forward in time.
- **Binary trust.** Every published trust record has weight +1. Distrust, strength of trust and trust evolution cannot be modelled, so the social regulariser treats all edges as equally informative.
- **Organic versus campaign adoption.** Observed ratings measure adoption that happened without a campaign. Simulated reach measures additional adoption caused by seeds. The two are compared only as a rank agreement, never as the same quantity.
- **Sampled negatives.** Ranking metrics use uniformly sampled unobserved items plus the held-out positives. This is standard but easier than scoring the whole catalogue, and a strong popularity baseline can exploit it.
- **Model assumptions.** IC and SIR assume one item, no competition, no re-exposure except in SIR, and a static network. Real campaigns compete for attention and users change their trust relations.
- **Group construction.** Conflict groups are synthetic by design (the most dissimilar users in a random pool). Real groups have a shared history that the dataset does not record.
- **Selection pool.** Greedy selection is restricted to a proxy-prefiltered candidate pool for tractability, so the reported reach is a lower bound on what an exhaustive search could achieve and an upper bound on what a production system with a tighter budget could afford.
- **Single dataset, single machine.** All results come from one Epinions core measured on one machine; absolute timings will differ elsewhere, although the scaling shapes should not.

10. Conclusion

The assignment asked for one system that recommends, explains itself socially, spreads an item, chooses whom to seed, serves groups and can be evaluated and controlled. Each requirement is implemented, measured and reported separately, and the same evaluation protocol and random seeds run through all of them.

Two findings stand out. First, the two recommender families are complementary: matrix factorisation is the better rating predictor (RMSE 1.0373) while item-kNN is the better ranker (NDCG@10 0.1384), so a system that has to choose what to put in front of a user should not be selected on RMSE alone. Second, the social signal is real but small and must be weighted carefully: at the selected beta = 0.5000 the graph comparison in Section 3 shows that trust and behavioural similarity are not interchangeable sources of evidence, and that any positive weight buys rating accuracy with ranking accuracy.

On the diffusion side, the campaign succeeds or fails on a knife edge of transmission probability, the three event types must be counted separately, and a campaign that percolates saturates quickly: informed seed selection beats random draws by up to 23.6% at the tightest budget and by nothing once the budget is generous, so the recommendation is to spend the first few seeds carefully rather than to spend many. The intervention study closes the loop: reach can be suppressed cheaply, but not without a measurable cost to the users the campaign was meant to serve, and the affinity gate is the better trade-off of the two levers tested.

Appendix A. Reproducibility

The environment is a uv-managed Python 3.11 project. The raw Epinions files are committed under `data/raw/epinions`, the processed core and every result are regenerated from the configuration below.

```
uv sync
uv run sntp prepare # build data/processed/epinions
uv run sntp run all # every experiment, figures and this PDF
uv run pytest -q # unit tests
```

Hardware: Windows-10-10.0.19045-SP0, 4 logical CPUs, 15.7 GB RAM, Python 3.11.9.

```
seed : 20261019
dataset core : min_user=10, min_item=10
```

```
folds : valid=0.1, test=0.1, seed=20261019
Bias-MF : k=32, lr=0.002, lambda=0.2, epochs<=40
Item-kNN : k=20, shrinkage=10.0, clip=0.5
training seeds : [11, 23, 37]
evaluation : K=10, negatives=100, relevance>=4.0
social betas : [0.0, 0.05, 0.1, 0.25, 0.5, 1.0, 2.0]
diffusion : p0=0.04, gamma=0.7, recovery=0.35, sims=100
influence : budgets=[5, 10, 20, 40], mc_select=20, mc_final=200
groups : size=5, n=60
```

Appendix B. Requirement to code map

requirement	implementation	command	report section
1. Environment representation	data.py, graph.py	sntp run dataset	Sections 1
2. Two recommenders	recommenders/bias_mf.py, recommenders/item_knn.py	sntp run recommenders	Section 2
3. Social extension	recommenders/social.py, graph.py	sntp run social	Section 3
4. Diffusion models	diffusion.py	sntp run diffusion	Section 4
5. Seed selection	seeds.py	sntp run influence	Section 5
6. Group recommendation	groups.py	sntp run groups	Section 6
7. Evaluation	metrics.py, evaluation.py	sntp run evaluation	Section 7
8. Intervention and cost	diffusion.py, experiments/rq8_intervention.py	sntp run intervention	Section 8

Table 33. Where each requirement is implemented and evaluated.